

# Data-Driven Fleet Operations: Demand Analytics and Smart Charging for Urban Ride-Hailing in Chicago

Advanced Analytics and Applications

Team 06



## Authors:

Louis Schreier (7381065)

Daniel Schumacher (7409287)

Timo Ercole (7408459)

**Contact:** student@smail.uni-koeln.de

**Supervisor:** Univ.-Prof. Dr. Wolfgang Ketter

**Co-Supervisor:** Janik Muires

Chair of Information Systems for Sustainable Society  
Faculty of Management, Economics and Social Sciences  
University of Cologne

2026-07-26

## Table of contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                              | <b>1</b>  |
| 1.1      | Business Context . . . . .                       | 1         |
| 1.2      | Business Goal and Data Mining Goal . . . . .     | 1         |
| <b>2</b> | <b>Data Description</b>                          | <b>1</b>  |
| 2.1      | Taxi Trip Data . . . . .                         | 1         |
| 2.2      | Weather Data . . . . .                           | 2         |
| 2.3      | Point-of-Interest Data . . . . .                 | 2         |
| <b>3</b> | <b>Data Preparation</b>                          | <b>2</b>  |
| 3.1      | Cleaning the Trip Data . . . . .                 | 2         |
| 3.2      | From Trips to a Forecasting Grid . . . . .       | 3         |
| <b>4</b> | <b>Descriptive Analytics</b>                     | <b>3</b>  |
| 4.1      | When Demand Happens . . . . .                    | 3         |
| 4.2      | Where Demand Happens . . . . .                   | 4         |
| 4.3      | Flows Between Areas . . . . .                    | 4         |
| <b>5</b> | <b>Data Analytics</b>                            | <b>5</b>  |
| 5.1      | Predictive Analytics . . . . .                   | 5         |
| 5.1.1    | Forecasting Problem and Baselines . . . . .      | 5         |
| 5.1.2    | Support Vector Machine . . . . .                 | 6         |
| 5.1.3    | Feedforward Neural Network . . . . .             | 6         |
| 5.1.4    | Model Comparison . . . . .                       | 7         |
| 5.2      | Smart Charging . . . . .                         | 7         |
| <b>6</b> | <b>Conclusions and Recommendations</b>           | <b>8</b>  |
| 6.1      | What the Analysis Shows . . . . .                | 8         |
| 6.2      | Recommendations for the Fleet Operator . . . . . | 9         |
| 6.3      | Private or Public Charging . . . . .             | 9         |
| 6.4      | Limitations . . . . .                            | 9         |
| 6.5      | Outlook . . . . .                                | 10        |
| <b>A</b> | <b>Data Dictionaries</b>                         | <b>11</b> |
| A.1      | Taxi Trip Data . . . . .                         | 11        |
| A.2      | Weather Data . . . . .                           | 11        |
| A.3      | Point-of-Interest Data . . . . .                 | 12        |
| <b>B</b> | <b>Data Preparation Detail</b>                   | <b>12</b> |
| B.1      | Missing Values . . . . .                         | 12        |
| B.2      | Consistency Checks . . . . .                     | 13        |
| B.3      | Outlier Removal . . . . .                        | 13        |
| B.4      | Cleaning Levels . . . . .                        | 13        |
| B.5      | Weather and POI Preparation . . . . .            | 14        |
| B.6      | Granularity and Sparsity . . . . .               | 14        |
| <b>C</b> | <b>Additional Descriptive Figures</b>            | <b>14</b> |
| <b>D</b> | <b>Model Tuning and Diagnostics</b>              | <b>16</b> |
| D.1      | SVR Kernel and Hyperparameter Search . . . . .   | 16        |
| D.2      | SVR Error Decomposition . . . . .                | 17        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| D.3      | SVR Performance Across Resolutions . . . . .    | 17        |
| D.4      | Neural Network Search and Diagnostics . . . . . | 17        |
| <b>E</b> | <b>Smart Charging Environment and Training</b>  | <b>18</b> |
| E.1      | Markov Decision Process Design . . . . .        | 18        |
| E.2      | Agent: Deep Q-Network . . . . .                 | 19        |
| E.3      | Training Behavior . . . . .                     | 19        |
| E.4      | Evaluation Distributions . . . . .              | 19        |
| E.5      | Example Episode . . . . .                       | 19        |

## List of Figures

|    |                                                                                                                                                                                                                                                     |    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Pickup demand heatmap cross-tabulated by hour of day and day of week. . . . .                                                                                                                                                                       | 3  |
| 2  | Choropleth maps of pickup demand aggregated at community area level and H3 resolutions 6, 7, and 8. . . . .                                                                                                                                         | 4  |
| 3  | Net flow (pickups minus dropoffs) by H3 hexagon, aggregated across the full observation period. . . . .                                                                                                                                             | 5  |
| 4  | Learned charging policy across SoC and time slot: preferred action (off / low / medium / high) per state. . . . .                                                                                                                                   | 8  |
| 5  | H3 hexagonal grid overlaid on Chicago community areas at resolutions 6, 7, and 8. . . . .                                                                                                                                                           | 14 |
| 6  | Distribution of trip duration, distance, fare, tips, total cost, and inter-trip idle time across the full dataset. Distributions are clipped at the 99th percentile so airport flat rates and exceptional idle gaps don't distort the axes. . . . . | 15 |
| 7  | Pickup demand by 4-hour time bucket, covering the full 24-hour cycle. . . . .                                                                                                                                                                       | 15 |
| 8  | Origin–destination matrix for the top 10 community areas by total trip volume. . . . .                                                                                                                                                              | 16 |
| 9  | Training and validation loss curves for the final neural network model. . . . .                                                                                                                                                                     | 18 |
| 10 | Training curves: episode reward, final SoC, cost and penalty over 5,000 training episodes (100-episode moving average). . . . .                                                                                                                     | 20 |
| 11 | Distributions over 1,000 evaluation episodes: total reward, final SoC, and cost vs. penalty, compared across DQN Agent, Always-Max, and Random. . . . .                                                                                             | 20 |
| 12 | Example evaluation episode: SoC, charging power, and cost per 15-minute slot from 2 p.m. to 4 p.m. . . . .                                                                                                                                          | 20 |

# 1 Introduction

## 1.1 Business Context

A German automotive manufacturer is launching a ride-hailing platform for US customers, operated entirely by an electric vehicle fleet. It's entering a mature market with no operational track record, so it needs data-driven guidance on how to run an electric taxi fleet in a major city.

Bane & Wayne Partners (BWP) was engaged to provide that guidance. This report presents the work of BWP's Data Science Team, which uses historical Chicago ride-hailing data to characterize demand, forecast it, and design a cost-efficient charging strategy.

## 1.2 Business Goal and Data Mining Goal

The goal is actionable intelligence for launching and running the fleet. The analysis answers three questions:

1. **Where and when is demand concentrated?** This drives fleet positioning and driver scheduling, and tells the client where to enter the market first.
2. **How accurately can future demand be forecast?** Better predictions cut idle time and energy waste, and raise driver earnings.
3. **How should charging be managed to minimize cost?** Charging timed to electricity prices and predicted demand can cut operating cost sharply.

The work is split into four tasks. Task 1 cleans the raw trips, discretizes them onto Uber H3 hexagons, and enriches them with weather data. Task 2 uses descriptive spatial analytics to surface demand hotspots and flow patterns. Task 3 trains two models to forecast demand per hexagon and time bucket: a Support Vector Machine and a feedforward neural network. Task 4 develops a reinforcement learning agent for smart-charging policies under stochastic energy demand and dynamic pricing.

Three data sources feed the analysis: historical Chicago taxi trips from the Chicago Data Portal, hourly weather from Open-Meteo, and OpenStreetMap points of interest for urban context. The main text sticks to findings and decisions. Data dictionaries, cleaning rules, and tuning details are in the appendix.

# 2 Data Description

Three data sources feed the analysis: Chicago taxi trip records, hourly weather, and OpenStreetMap points of interest. Together they cover demand, environmental context, and urban structure. Full data dictionaries are in Section [A](#).

## 2.1 Taxi Trip Data

Downloaded from the Chicago Data Portal on 10 May 2026, the dataset holds every recorded taxi trip from January 2024 to early May 2026: **14,985,679 trips** across 21 columns, extended by seven derived temporal features at load time.

Three portal design choices matter downstream. Driver and trip IDs are SHA-1 hashes, so there's no personally identifiable information. All timestamps are rounded to the nearest 15 minutes for privacy, which caps temporal precision but doesn't affect hourly or 4-hourly aggregation. And location is reported only at census-tract level. With roughly 800 tracts in

Chicago the spatial grain is coarser than GPS, but each tract’s centroid coordinates support spatial joins.

## 2.2 Weather Data

Hourly weather comes from the Open-Meteo Historical Archive API (Open-Meteo, 2024) over the same date range. Chicago spans about 600 km<sup>2</sup> with real microclimatic differences between the lakefront, the urban core, and the far northwest, so a single weather station won’t represent it. We defined three weather zones by K-means clustering on all pickup coordinates, placing zone centroids where demand is densest: the downtown core, O’Hare, and the South Side. The result is **61,335 hourly observations** (~20,445 per zone) across nine variables. Temperatures range from  $-26.8\text{ }^{\circ}\text{C}$  to  $36.3\text{ }^{\circ}\text{C}$ , with apparent lows of  $-32.9\text{ }^{\circ}\text{C}$  from wind chill. That’s Chicago’s continental climate at work, and a plausible driver of taxi demand.

## 2.3 Point-of-Interest Data

POI data comes from OpenStreetMap via `osmnx` (Boeing, 2017). Raw tags (restaurant, bus\_stop, hospital, ...) are too fine-grained and overlapping to use directly, so we collapsed them into seven demand-relevant categories, from food & nightlife (5,640 locations) down to transport (135), leaving **9,124 POIs** after reducing polygons to centroids. POIs are static: they don’t vary over time and aren’t counted as demand. Aggregated per hexagon as category counts, they proxy how strongly each cell attracts trips.

# 3 Data Preparation

## 3.1 Cleaning the Trip Data

Public portal data needs validation before use. Rather than applying fixed rules upfront, we worked through the dataset issue by issue, diagnosing each cause and choosing a targeted fix: missing fields, “ghost trips” with zero duration and distance, zero-distance meter cancellations, and physically implausible speeds and fares. The full rule set with per-rule counts is in Section B. Cleaning removes just over 10 % of raw records. It doesn’t distort the geography either: per-neighborhood pickup counts before and after cleaning correlate at 1.0000.

One finding matters for revenue analysis. In 47.9 % of rows the billed total exceeds the sum of its components, almost always by exactly \$0.50, concentrated in card and mobile payments. That pattern points to an unitemized card surcharge. We apply no correction and treat `trip_total` as the authoritative revenue figure.

Not every task needs the same rigor. A trip with an odd speed or fare is still a real demand event, so **demand forecasting** uses a minimal cleaning level that keeps 99.99 % of rows, while **trip-level analyses** such as fare and duration use the full level, keeping 89.99 %.

Weather and POI data needed lighter preparation: a redundant weather condition code was dropped, apparent gaps at the spring clock change were verified as correct rather than missing, and POI geometries were reduced to single coordinates and mapped to the seven categories (details in Section B).

### 3.2 From Trips to a Forecasting Grid

The models don't run on individual trips. Trips are aggregated to a grid, one row per (*hexagon, time bucket*) with the pickup count as the target. Cells with no pickups are kept as explicit zero-demand rows, so the models learn from quiet periods too.

Grain is a business trade-off: finer cells localize demand better but leave more of the grid empty and harder to forecast. We operate at **H3 resolution 6 with 4-hour buckets**: 27 hexagons  $\times$  5,107 time buckets = 137,889 rows at a manageable 12.6 % zero-demand share. One step finer pushes the zero-demand share toward 50 % without a matching gain in operational precision (full sparsity comparison in Section B). Section 5 assesses how coarser and finer grids affect model accuracy.

Every model input is knowable before the bucket begins: calendar context (hour, day of week, month, weekend and US-holiday flags, encoded as sine-cosine pairs so that 23:00 sits next to 00:00), the nine weather variables for the nearest zone, and a per-hexagon indicator that lets each cell learn its own baseline. Together these give 51 input variables per grid cell.

## 4 Descriptive Analytics

### 4.1 When Demand Happens

Taxi demand in Chicago follows predictable rhythms tied to work schedules, social activity, and weather. Understanding those rhythms is the starting point for fleet positioning and driver scheduling.

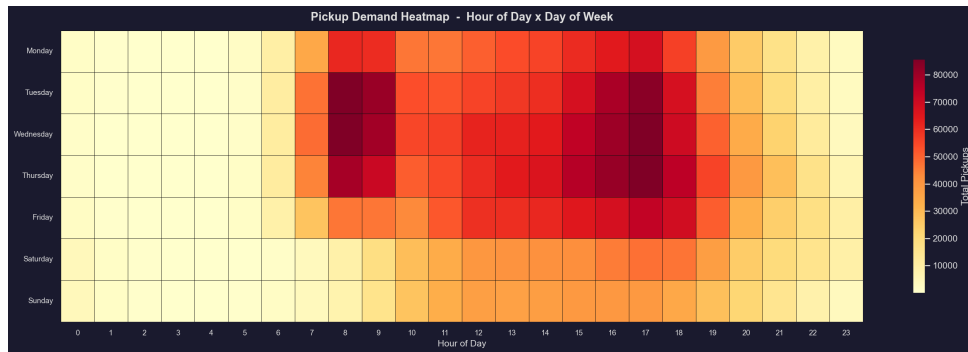


Figure 1: Pickup demand heatmap cross-tabulated by hour of day and day of week.

Weekdays are bimodal: a morning commuter peak around 08:00 and a stronger, broader evening peak between 16:00 and 19:00, with Tuesday to Thursday carrying the highest volumes in the dataset. Weekends run on a different clock. Demand starts later, builds gradually through the afternoon, and shows no commuter spike, because this is leisure travel. The 00:00–06:00 window is uniformly quiet on every day of the week, which makes it the natural slot for charging and maintenance in an electric fleet.

Individual trips are short urban hops: median duration around ten minutes, most trips under five miles, and fares to match, with a long tail from airport and cross-city rides (distributions in Section C). Idle time between a driver's consecutive trips concentrates in the 0–30 minute range, though a mean near an hour points to periods of deliberate waiting or repositioning.

## 4.2 Where Demand Happens

To analyze space consistently we discretize the city with Uber’s H3 hexagonal grid, which offers uniform cell areas and clean adjacency where administrative boundaries don’t (grid illustration in Section C). The maps below use resolution 7. The forecasting models in Section 5 operate one step coarser, at resolution 6.

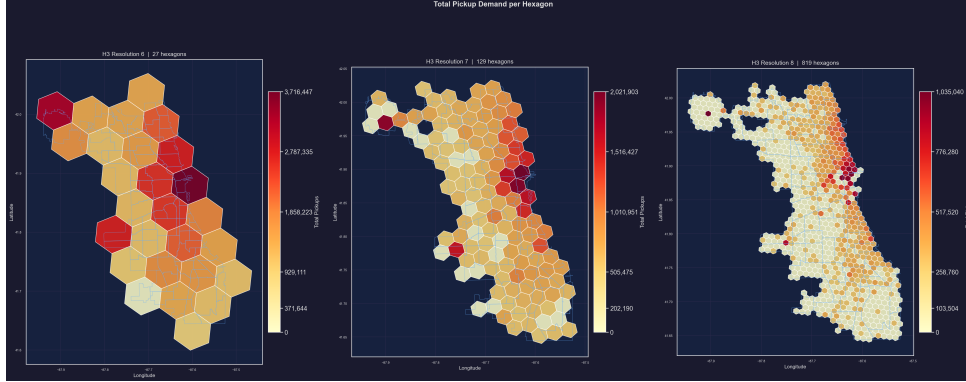


Figure 2: Choropleth maps of pickup demand aggregated at community area level and H3 resolutions 6, 7, and 8.

The Near North Side and the Loop dominate demand, consistent with their role as Chicago’s central business and entertainment district, while O’Hare forms a structurally distinct cluster in the far northwest. As resolution increases the hotspots sharpen into specific corridors (the Magnificent Mile, River North, the area around O’Hare), and areas that look moderate at community level turn out to vary strongly inside. Community-area aggregation hides exactly the sub-neighborhood differences a fleet needs for positioning decisions.

The picture shifts over the day, but the downtown-lakefront corridor stays dominant in every 4-hour window (maps in Section C). Overnight, the downtown core keeps relative activity from hospitality traffic. Early mornings add a secondary hotspot around Midway from first departures, and the evening windows intensify the core sharply while the outskirts stay comparatively quiet.

## 4.3 Flows Between Areas

Where trips go matters as much as where they start. The origin–destination analysis (matrix in Section C) shows the Near North Side and the Loop dominating as both origins and destinations. O’Hare, despite its distance, sends nearly 500,000 trips toward those two areas alone, making the airport-to-downtown corridor one of the most dependable sources of paid mileage in the network. Vehicles dropping passengers in outer neighborhoods, by contrast, face a structural repositioning problem to get back to demand.

Net flow, defined as pickups minus dropoffs per hexagon, turns that imbalance into something a planner can act on: persistently positive cells need more vehicles than dropoffs return, and negative cells accumulate them.

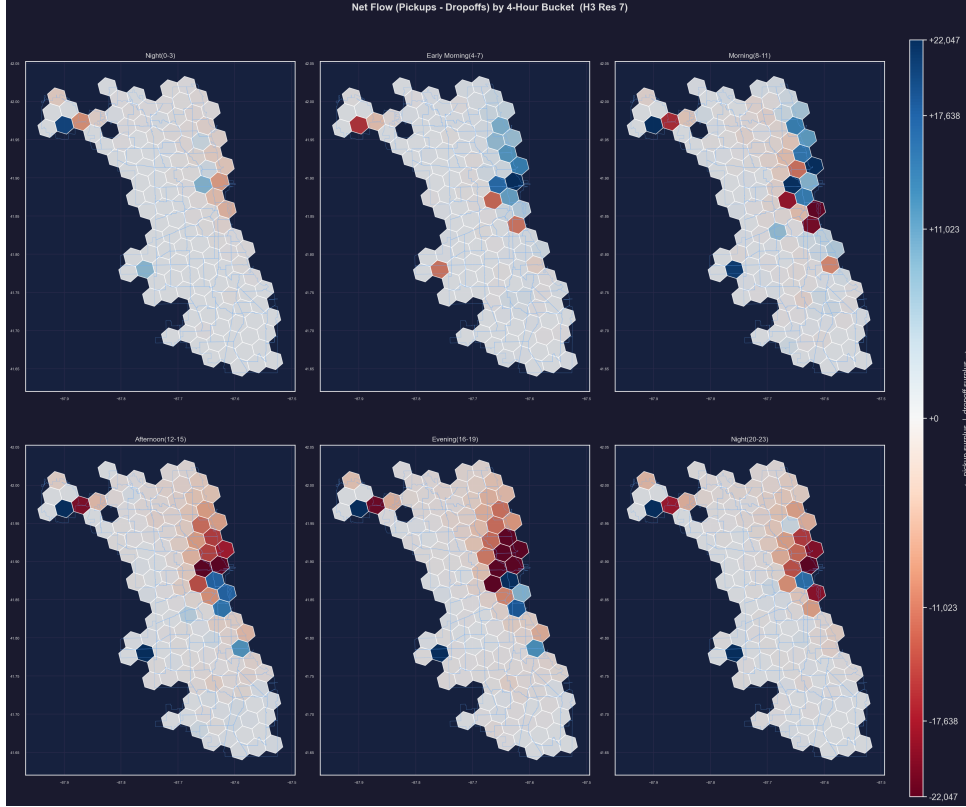


Figure 3: Net flow (pickups minus dropoffs) by H3 hexagon, aggregated across the full observation period.

The downtown core and the O’Hare corridor consistently hand out more trips than they take back, while the residential outskirts act as vehicle sinks. For an electric fleet this points straight at infrastructure: chargers placed in the negative-flow residential zones turn otherwise idle accumulation into productive charging time, before vehicles are repositioned to the high-demand corridors.

## 5 Data Analytics

### 5.1 Predictive Analytics

#### 5.1.1 Forecasting Problem and Baselines

The task is to predict **taxi demand per hexagon and 4-hour window**, using only information available before the window begins: time of day, day of week, weather forecast, and location. A dispatcher scheduling tomorrow’s shifts doesn’t have tomorrow’s actual counts, so the models use no lagged demand. Trip counts are heavily skewed (a few downtown cells see hundreds of pickups per window, most outer cells single digits), so we train on log-transformed counts and report all metrics back on the trip scale. The grid is randomly split 50 / 20 / 30 into training, validation, and test data. The test set is touched once, for the final numbers.

Two naive benchmarks give the results business meaning. The *per-cell mean*, the historical average per hexagon and window, scores MAE 67.18 trips. *Climatology*, the average per hexagon, hour, and weekday-vs-weekend, scores MAE 25.63. Climatology is the hurdle that matters. It’s what an operator gets from a simple lookup schedule, so a model earns its complexity only by beating it.



### 5.1.2 Support Vector Machine

We tuned a Support Vector Machine for regression across kernel types and regularization settings on a validation subsample; the search and its intermediate results are in Section D. The best configuration, an RBF kernel with  $C = 10$ , confirms that the link between weather, calendar, and demand is non-linear. Table 1 shows the final test-set results on 41,367 grid cells.

Table 1: SVR test-set performance compared to baselines.

| Model         | MAE          | RMSE         | $R^2$        |
|---------------|--------------|--------------|--------------|
| Per-cell mean | 67.18        | 216.97       | 0.633        |
| Climatology   | 25.63        | 93.19        | 0.932        |
| SVR (RBF)     | <b>24.33</b> | <b>88.05</b> | <b>0.940</b> |

The SVR beats both baselines, but its margin over climatology is thin: MAE 24.33 against 25.63, roughly 5 %. Without access to recent actual counts, most of what it can learn is the seasonal pattern climatology already encodes. The 5 % it adds comes from weather and calendar interactions a fixed schedule can’t represent. The error is also unevenly distributed. The model is accurate in the many low-demand cells but under-predicts the busy downtown and airport hexagons where forecasts matter most, a known artifact of training on compressed log counts (decomposition in Section D).

We also tested the tuned model across coarser and finer grids, since the granularity choice itself is a business decision. Finer cells cut the absolute error per cell (from 73 to 31 trips MAE at 4-hour windows) simply because each cell holds fewer trips, while explained variance stays around 0.9; the full sweep is in Section D. Resolution 6 with 4-hour windows remains the working choice. Census tracts (roughly 800 units instead of 27) would multiply sparsity without operational benefit.

### 5.1.3 Feedforward Neural Network

We trained a feedforward neural network on the same grid and features, so the two models are directly comparable. A grid search over architectures, learning rates, and optimizers (detailed in Section D) selected a network with two hidden layers of 256 neurons, trained with the Adam optimizer. Table 2 shows the final test-set results.

Table 2: Neural network test-set performance compared to the climatology baseline.

| Model          | MAE          | RMSE         | $R^2$        |
|----------------|--------------|--------------|--------------|
| Climatology    | 24.42        | 87.22        | 0.937        |
| Neural network | <b>21.16</b> | <b>80.78</b> | <b>0.946</b> |

The network also beats climatology, and by a wider margin: MAE 21.16 against 24.42, a 13 % improvement. It captures more of the demand structure beyond the weekly rhythm, in particular how weather, time, and location interact. A residual bias of  $-8.25$  trips shows the extreme downtown peaks are still partly missed. Those spikes come from events the data can’t see, such as concerts and sports fixtures, and would need an event calendar to forecast reliably.

The model’s operational output is an on-demand forecast for any combination of location, time, weather, and day type. Queried for an outer-city hexagon on a rainy Sunday in July, it

predicts demand rising from 34.4 trips at 10:00 to a peak of 37.3 at midday, then easing to 20.4 by 18:00. That’s about 282 trips across the window, following the leisure-weekend build-up seen in the descriptive analysis.

#### 5.1.4 Model Comparison

Both models run on the same resolution 6 grid with 4-hour windows, each measured against its own climatology baseline (the notebooks use different random splits, hence the slightly different baseline values):

Table 3: Each model’s performance relative to the climatology baseline.

| Model          | Setting                   | Climatology<br>MAE | Model MAE | Improvement           |
|----------------|---------------------------|--------------------|-----------|-----------------------|
| SVR (RBF)      | H3 res. 6, 4 h<br>windows | 25.63              | 24.33     | <b>Better</b> (+5 %)  |
| Neural network | H3 res. 6, 4 h<br>windows | 24.42              | 21.16     | <b>Better</b> (+13 %) |

The neural network wins for two structural reasons. It trains on the full dataset while the SVR, which scales poorly, had to learn from a small subsample. And it learns feature interactions jointly, where the SVR applies one fixed transformation to every input. Both beat the seasonal schedule, but the network improves on it by roughly two and a half times as much. For an operator positioning vehicles under changing weather or on atypical weekdays, that responsiveness is the point: a seasonal average says nothing about whether tomorrow’s rainy Monday will run busier than a dry one.

## 5.2 Smart Charging

Forecasting tells the operator where demand will be. Charging determines what it costs to serve. We model a single electric taxi that arrives home at 2 p.m. and must leave the garage at 4 p.m. Instead of charging at a flat rate, an agent sets the charging power every 15 minutes, balancing two goals: keep the recharging cost low and guarantee enough energy for the upcoming shift, whose demand is unknown while charging decisions are made. Because the decisions are sequential and the outcome uncertain, we frame this as a reinforcement learning problem and train a Deep Q-Network (DQN). The agent observes the battery’s state of charge (SoC) and the time slot, chooses among four power levels (off, 7.3, 14.7, or 22 kW), pays a charging cost that grows steeply with power, and takes a large penalty if the battery falls short of the realized shift demand. The full environment design, agent setup, and training diagnostics are in Section E.

The learned policy (Figure 4) is simple enough to hand to an operations team: charge hard while the battery is low, taper as a buffer builds, switch off once SoC comfortably exceeds expected demand. Because a shortfall is penalized far more heavily than expensive charging, the policy is risk-averse by construction. It targets a small surplus above the 30 kWh mean demand rather than the mean itself.

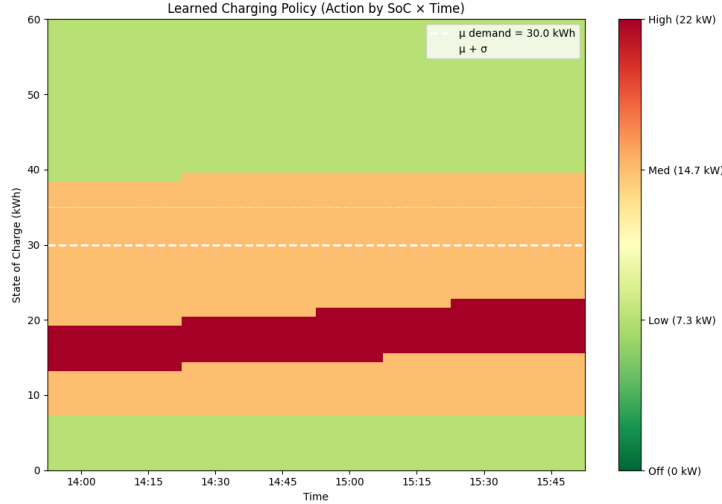


Figure 4: Learned charging policy across SoC and time slot: preferred action (off / low / medium / high) per state.

Table 4: Policy comparison over 1,000 evaluation episodes.

| Policy     | Avg. Reward   | Avg. Cost    | Avg. Penalty | Avg. Final SoC (kWh) | Shortfall rate |
|------------|---------------|--------------|--------------|----------------------|----------------|
| DQN Agent  | <b>-65.10</b> | 64.45        | 0.65         | 42.8                 | <b>0.4 %</b>   |
| Always-Max | -86.99        | <b>86.99</b> | 0.00         | 56.5                 | 0.0 %          |
| Random     | -202.91       | 56.39        | 146.52       | 34.4                 | 29.1 %         |

Evaluated over 1,000 episodes (Table 4), the agent delivers nearly the safety of always charging at maximum, with a far better cost profile. Always-Max never runs short but pays roughly 35 % more in charging cost for a buffer it doesn’t need, ending with an average of 56.5 kWh against a 30 kWh mean demand. The random policy shows the other failure mode: a 29 % shortfall rate would mean roughly one in three shifts starting under-charged. The DQN agent runs short in 0.4 % of episodes, and that reliability level is tunable through the shortfall penalty.

For the client, the takeaway is that “charge to full” is not cost-optimal even though it feels safest. The learned policy could be deployed as a plain lookup table on a home charging unit, since the state space is only two-dimensional. Two caveats belong in the client conversation: the agent is trained on a fixed demand distribution and needs retraining if trip volumes shift, and the shortfall penalty is a dial to set according to the operator’s actual tolerance for an under-charged vehicle, not a fixed technical constant.

## 6 Conclusions and Recommendations

### 6.1 What the Analysis Shows

Demand in Chicago is concentrated in space and structured in time. A handful of hexagons account for a disproportionate share of all pickups: the downtown core, the Magnificent Mile, the O’Hare corridor. Most of the city sits close to zero for much of the day. On top of that sits a predictable weekly rhythm: bimodal weekdays with a strong evening peak, later and flatter weekends, and a quiet overnight window common to every day.

Both forecasting models beat the seasonal climatology baseline, the neural network by a clear margin (MAE 21.16 against 24.42) and the SVM only just. The reinforcement learning agent learned a charging policy that reliably met the driver’s departure energy needs at a cost well below a naive charge-at-maximum strategy.

## 6.2 Recommendations for the Fleet Operator

**Position vehicles for the corridors, not the map as a whole.** Demand lives in a handful of places. That argues for a phased rollout starting in the downtown-to-lakefront corridor and the airport connections, not an even spread across all 77 community areas. O’Hare alone sends close to half a million trips toward the Near North Side and the Loop, so the airport-to-downtown axis is a dependable source of paid mileage from day one.

**Use the net-flow signal to plan repositioning.** Pickups and dropoffs don’t balance locally. Downtown and the airport hand out more trips than they take back, while the residential edges absorb vehicles without generating return demand, and a fleet that ignores this accumulates idle cars in the wrong places. Repositioning logic should be built around the net-flow map, treating the outer sinks as places to hold and charge rather than places to wait for fares.

**Forecast for planning, and know what the forecast can’t see.** The neural network earns its complexity because it responds to weather and atypical days in a way a fixed schedule can’t. It’s the right tool for shift planning and for questions like expected demand in a given area on a rainy Sunday afternoon. What it doesn’t capture are one-off spikes from concerts, fixtures, and conventions. Treat model output as a strong baseline to be overridden by hand on known event days, not as a complete demand picture.

## 6.3 Private or Public Charging

The charging economics and the demand geography point to a mixed answer. The cost function penalizes high charging power convexly, so charging slowly over a long window is much cheaper than short bursts at full rate. The agent exploited exactly this, spreading energy across the available slots and holding a modest buffer above expected demand. That favors private or depot charging for the bulk of the fleet’s energy: it lines up with the quiet overnight window, draws at a low and cheap rate for hours, and can sit in the negative-net-flow residential zones where cars would otherwise idle.

Public fast charging works against this cost structure, because its whole value is delivering energy quickly, which is the expensive corner of the curve. Our recommendation: anchor the fleet on private or depot charging near where vehicles naturally end their shifts, and use public fast charging only as an opportunistic top-up for a car that would otherwise have to leave a high-demand corridor to recharge.

## 6.4 Limitations

Several caveats bound these findings. Locations are reported at census-tract centroids rather than true GPS, so hotspot detection and flow maps inherit a coarse spatial grain, and fine within-tract patterns are invisible by construction. The forecasting models see no recent demand counts and no event calendar, which is why both under-predict the largest downtown spikes and why the SVR barely improves on a seasonal average. The SVR also had to be tuned on a small subsample because it doesn’t scale to the full grid. And the charging agent is trained against a stylised environment with a fixed cost coefficient and normally distributed departure demand.

Real prices and energy needs vary, so the learned policy is a proof of concept, not a deployable controller.

## 6.5 Outlook

The most valuable next step is adding lagged demand to the forecasts. The previous period's actual count is usually the strongest single predictor, and its absence is the clearest gap between this setup and a production system. A spatially aware hexagon encoding (distance to center, proximity to transport hubs, neighboring demand) would let the models generalize across cells, and an event calendar would target precisely the spikes they currently miss. On the charging side, real day-ahead electricity prices would let the agent optimize against the tariffs a fleet actually faces. Beyond that, pairing the demand maps with fare-per-mile and idle-time patterns would show where drivers earn most efficiently, and an explicit repositioning model would turn the net-flow findings into concrete vehicle movements.

## A Data Dictionaries

### A.1 Taxi Trip Data

Table 5 lists the key fields of the trip dataset. Seven temporal features (date, hour, day of week, month, year, week, weekend flag) are derived at load time, bringing the working total to 28 columns.

Table 5: Core fields of the Chicago taxi trip dataset.

| Column                   | Type             | Description                                       |
|--------------------------|------------------|---------------------------------------------------|
| trip_id                  | string           | SHA-1 hash; unique journey identifier             |
| taxi_id                  | string           | SHA-1 hash; anonymized vehicle identifier         |
| trip_start_timestamp     | datetime (UTC−6) | Rounded to the nearest 15 minutes                 |
| trip_end_timestamp       | datetime (UTC−6) | Rounded to the nearest 15 minutes                 |
| trip_seconds             | float            | Recorded journey duration in seconds              |
| trip_miles               | float            | Recorded journey distance in miles                |
| pickup_census_tract      | float            | Census tract of pickup ( $\approx 54\%$ missing)  |
| dropoff_census_tract     | float            | Census tract of dropoff ( $\approx 56\%$ missing) |
| pickup_community_area    | float            | Community area of pickup (0.05% missing)          |
| dropoff_community_area   | float            | Community area of dropoff (7.2% missing)          |
| fare                     | float            | Metered fare in USD                               |
| tips                     | float            | Tip amount in USD                                 |
| trip_total               | float            | Total charge billed to the passenger              |
| payment_type             | string           | Credit card, cash, mobile, etc.                   |
| company                  | string           | Taxi company name                                 |
| pickup_centroid_lat/lon  | float            | Centroid of the pickup census tract               |
| dropoff_centroid_lat/lon | float            | Centroid of the dropoff census tract              |

### A.2 Weather Data

The three weather zones are centered on the downtown core (41.886°N, 87.629°W), the O'Hare area (41.979°N, 87.903°W), and the South Side (41.785°N, 87.750°W). Both the elbow method and the silhouette score pointed to  $k = 3$ . Table 6 lists the hourly variables fetched for each zone.

Table 6: Hourly weather variables fetched from Open-Meteo.

| Variable             | Unit | Description                                      |
|----------------------|------|--------------------------------------------------|
| temperature_2m       | °C   | Air temperature at 2 m height                    |
| apparent_temperature | °C   | Feels-like temperature (wind chill / heat index) |
| precipitation        | mm   | Total precipitation                              |
| rain                 | mm   | Liquid precipitation                             |
| snowfall             | cm   | Snowfall accumulation                            |
| snow_depth           | cm   | Snow depth on ground                             |
| windspeed_10m        | km/h | Wind speed at 10 m                               |
| windgusts_10m        | km/h | Wind gust speed at 10 m                          |
| cloud_cover          | %    | Fractional cloud cover                           |

### A.3 Point-of-Interest Data

The OSM query covers four tag categories (**amenity**, **shop**, **tourism**, **leisure**). Raw tags are collapsed into seven demand-relevant categories via a lookup table, grouping tags with similar expected demand timing and dropping types too niche to matter (Table 7).

Table 7: POI counts by functional category (OpenStreetMap, Chicago).

| Category         | Count | Examples                                     |
|------------------|-------|----------------------------------------------|
| Food & Nightlife | 5,640 | Restaurants, bars, cafés, fast food          |
| Education        | 1,219 | Schools, universities, libraries             |
| Shopping         | 1,084 | Supermarkets, clothing, electronics          |
| Healthcare       | 469   | Hospitals, clinics, pharmacies               |
| Entertainment    | 348   | Theaters, cinemas, museums                   |
| Accommodation    | 229   | Hotels, hostels                              |
| Transport        | 135   | Bus stops, subway entrances, ferry terminals |

## B Data Preparation Detail

### B.1 Missing Values

Different fields are missing for different reasons, so each gets its own treatment (Table 8).

Table 8: Missing-value handling by field.

| Field                   | Missing  | Cause                                                                               | Treatment                                                                        |
|-------------------------|----------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Timestamps              | < 0.01 % | Fall in the autumn DST fold, where one hour repeats and the occurrence is ambiguous | Remove                                                                           |
| Drop-off community area | 7.2 %    | Drop-off outside the 77 official community areas (suburban)                         | Recover via spatial join where inside the city; retain suburban trips unassigned |

| Field         | Missing   | Cause                      | Treatment                                                       |
|---------------|-----------|----------------------------|-----------------------------------------------------------------|
| Trip duration | small     | Not reported but derivable | Impute from end – start timestamp                               |
| Trip distance | small     | Not reported               | Estimate with great-circle distance (understates road distance) |
| Vehicle ID    | 8 records | Unattributable to any taxi | Remove                                                          |

## B.2 Consistency Checks

Some records are present but aren’t real journeys. **Ghost trips** have zero seconds, identical start and end time, and identical start and end location, which looks like a meter switched on and off. All four conditions must hold, so genuine short trips survive; we remove the **153,078** records that qualify (~1 %). A further **1,014,515 trips** (6.8 %) log zero distance despite positive duration, with a median of 46 seconds and a quarter under 6 seconds. These are overwhelmingly meter cancellations or GPS failures rather than real journeys, and all are excluded. We also confirmed there are no end-before-start inversions and no duplicate trip IDs.

## B.3 Outlier Removal

After dropping non-journeys, we remove physically implausible values using two derived metrics, average speed and fare per mile (Table 9).

Table 9: Outlier removal criteria applied to the taxi trip dataset.

| Criterion                    | Records removed | Share of dataset |
|------------------------------|-----------------|------------------|
| Non-positive fare            | 5,243           | 0.04 %           |
| Distance exceeding 100 miles | 960             | 0.01 %           |
| Duration exceeding 3 hours   | 11,629          | 0.08 %           |
| Average speed above 70 mph   | 6,663           | 0.05 %           |
| Average speed below 2 mph    | 155,478         | 1.13 %           |
| Fare below \$2 per mile      | 28,537          | 0.21 %           |
| Fare above \$100 per mile    | small           | < 0.01 %         |

## B.4 Cleaning Levels

Table 10: Rows retained under each cleaning level.

| Level            | What is removed                                                                           | Rows retained        |
|------------------|-------------------------------------------------------------------------------------------|----------------------|
| Demand (minimal) | DST-ambiguous timestamps                                                                  | 14,984,551 (99.99 %) |
| Trip (full)      | Above, plus missing vehicle ID, ghost trips, zero-distance trips, and speed/fare outliers | 13,485,728 (89.99 %) |



## B.5 Weather and POI Preparation

**Condition codes.** The raw weather data carries a WMO condition code (0–99). Treating it as a number implies a false ordering (code 95 isn’t “95× worse” than code 1), and only three conditions actually appear: clear, rain, snow. Since rain and snow already have their own numeric columns, the code adds nothing and is dropped.

**Clock-change gaps.** Each zone shows a one-hour gap on spring-forward days (2024-03-10, 2025-03-09). The 02:00 hour doesn’t exist when clocks jump 01:00 → 03:00, so the gap is correct and no imputation is needed.

**POI geometry.** The OSM query returns points, polygons, and lines. All non-point features are reduced to their centroid, so every POI is a single coordinate. Each is mapped to one of seven functional categories via a lookup table, with unmapped types discarded; the final 9,124 POIs are assigned to the hexagon containing their coordinate.

## B.6 Granularity and Sparsity

Grain sets a trade-off: finer units capture local variation but leave more cells at or near zero, which is harder to forecast. Table 11 shows the effect.

Table 11: Zero-demand share across spatial and temporal granularities.

| Spatial unit                     | Time bucket | Grid cells | Zero-demand share |
|----------------------------------|-------------|------------|-------------------|
| H3 resolution 5 (large hexagons) | 1 hour      | 163,400    | 5.8 %             |
| H3 resolution 6                  | 1 hour      | 551,475    | 26.8 %            |
| H3 resolution 6                  | 4 hours     | 137,889    | 12.6 %            |
| H3 resolution 7                  | 4 hours     | 653,696    | 49.2 %            |

## C Additional Descriptive Figures

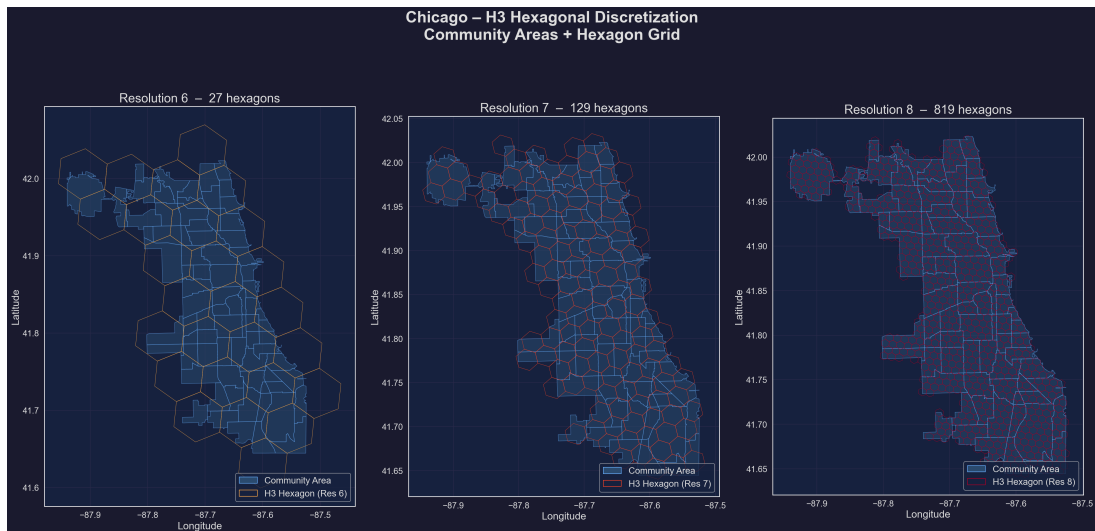


Figure 5: H3 hexagonal grid overlaid on Chicago community areas at resolutions 6, 7, and 8.

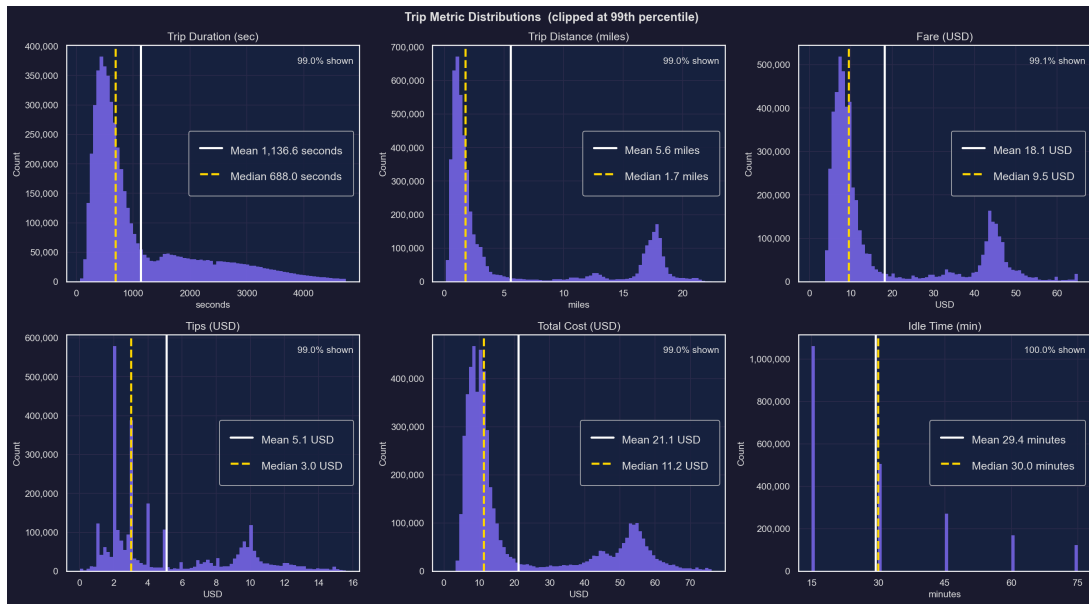


Figure 6: Distribution of trip duration, distance, fare, tips, total cost, and inter-trip idle time across the full dataset. Distributions are clipped at the 99th percentile so airport flat rates and exceptional idle gaps don't distort the axes.

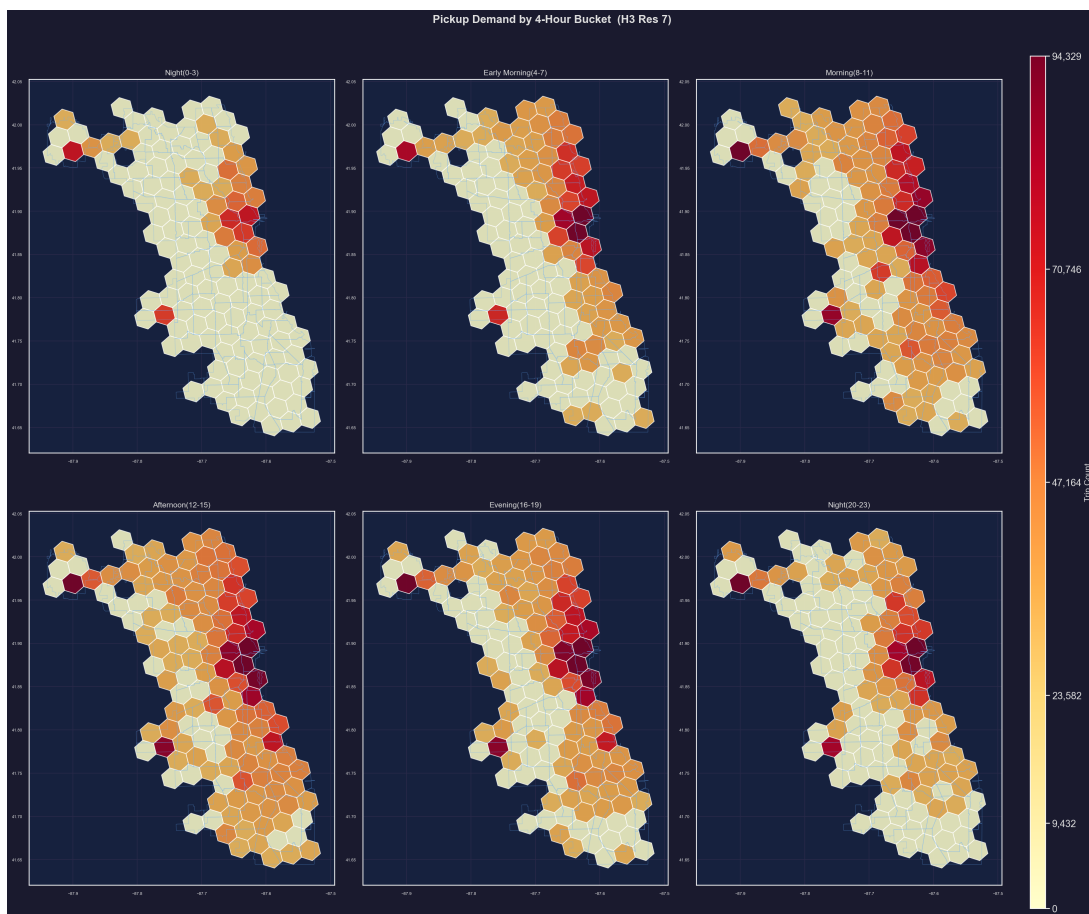


Figure 7: Pickup demand by 4-hour time bucket, covering the full 24-hour cycle.

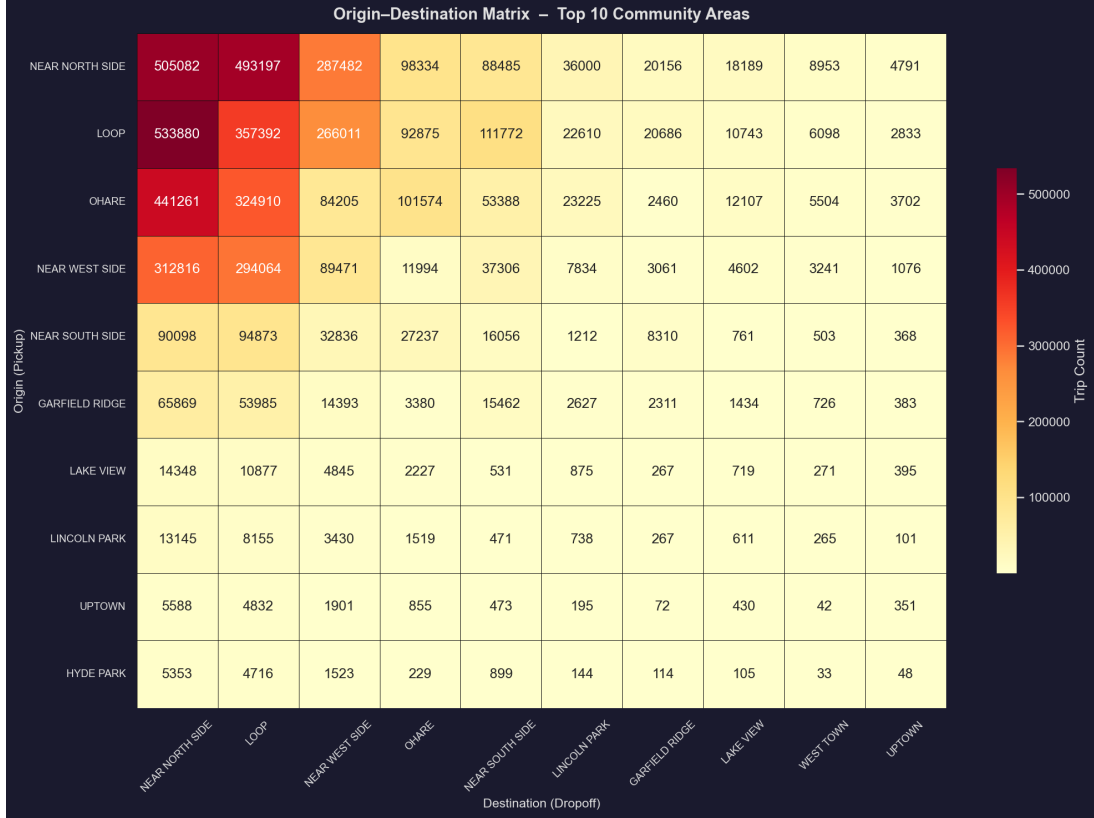


Figure 8: Origin-destination matrix for the top 10 community areas by total trip volume.

## D Model Tuning and Diagnostics

### D.1 SVR Kernel and Hyperparameter Search

A Support Vector Machine for regression (SVR) fits a function that stays within a tolerance band around the observed values while remaining as smooth as possible. The kernel sets the shape of that function: linear allows only straight-line relationships, polynomial curved ones, and the Radial Basis Function (RBF) highly flexible non-linear relationships. We start simple and add complexity, so that any gain can be attributed to the added flexibility rather than assumed.

Three settings are tuned alongside the kernel:  $C$ , how closely the model fits the training data; epsilon, the width of the no-penalty tolerance band; and gamma, how far each observation's influence reaches (RBF and polynomial only). SVR scales poorly with training set size, so tuning ran on a random subsample of 8,000 observations, with the best configuration refit on 30,000 rows for final evaluation.

Table 12: Best SVR configuration per kernel type, scored on the validation set.

| Kernel type | Best regularization | Validation MAE     |
|-------------|---------------------|--------------------|
| Linear      | $C = 1.0$           | 39.58 trips        |
| Polynomial  | $C = 10.0$          | 29.59 trips        |
| RBF         | $C = 10.0$          | <b>25.51 trips</b> |

Moving from linear (MAE 39.58) to RBF (MAE 25.51) shows the value of flexibility: the

link between weather, calendar and demand is non-linear, and the RBF kernel captures that curvature. The RBF kernel with  $C = 10$  and  $\epsilon = 0.1$  goes into final evaluation.

## D.2 SVR Error Decomposition

Table 13: SVR test-set error decomposed by demand level.

| Demand level (trips per window) | MAE    | Observations |
|---------------------------------|--------|--------------|
| 0–1                             | 0.68   | 7,580        |
| 1–5                             | 1.70   | 7,035        |
| 5–20                            | 3.81   | 10,328       |
| 20–100                          | 11.91  | 8,725        |
| 100+                            | 109.89 | 7,699        |

## D.3 SVR Performance Across Resolutions

Table 14: SVR performance across spatial and temporal resolutions.

| Hexagon size                    | Time window | MAE   | $R^2$ |
|---------------------------------|-------------|-------|-------|
| Large (resolution 5, 8 cells)   | 1 hour      | 24.57 | 0.860 |
| Large (resolution 5, 8 cells)   | 4 hours     | 73.37 | 0.938 |
| Medium (resolution 6, 27 cells) | 1 hour      | 8.18  | 0.889 |
| Medium (resolution 6, 27 cells) | 4 hours     | 31.10 | 0.902 |

Finer spatial units lower the absolute error because each hexagon holds fewer trips.  $R^2$  does not move monotonically: going from resolution 5 to 6 raises it at hourly windows (0.860 to 0.889) but lowers it at 4-hour windows (0.938 to 0.902).

## D.4 Neural Network Search and Diagnostics

The grid search covers hidden layer sizes ([64, 64], [128, 64], [256, 256] and [64, 64, 64, 64]), dropout (0 %, 10 %), learning rate (0.1, 0.01, 0.001) and optimizer (SGD, Adam). Each candidate trains for up to 200 iterations, stopping early after 20 iterations without validation improvement. The winning configuration (two hidden layers of 256 neurons each, no dropout, learning rate 0.001, Adam) is retrained from scratch on the combined training and validation data.

Holding that architecture fixed, we compare three activation functions (Table 15). Tanh edges out ReLU, and both are far ahead of sigmoid, which never trains to a usable error level because of gradient saturation: its neurons produce near-constant outputs across much of the input range, which stalls learning in deeper networks. The final model uses ReLU; the gap to tanh is under two trips of MAE, small enough to fall within run-to-run variation on a single validation split.

Table 15: Validation error by activation function for the best network architecture.

| Activation function | Validation MAE     |
|---------------------|--------------------|
| ReLU                | 21.93 trips        |
| Tanh                | <b>20.04 trips</b> |
| Sigmoid             | 93.29 trips        |

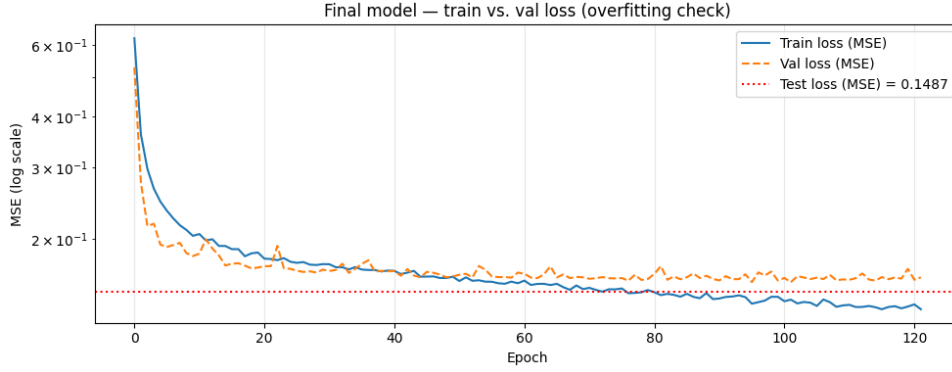


Figure 9: Training and validation loss curves for the final neural network model.

## E Smart Charging Environment and Training

### E.1 Markov Decision Process Design

**State space.** The state is a 2-dimensional vector: the current SoC in kWh, and the current 15-minute slot index, normalized to  $[0, 1]$ . Two components are sufficient because the charging window is fixed (2 hours, 8 slots) and the only quantity that evolves within it is the battery level; calendar or weather features would be redundant, since none of them change within a single 2-hour window. Keeping the state low-dimensional also keeps the learned policy interpretable as a simple heatmap over SoC and time slot (Figure 4), which matters for explaining the agent’s behavior to the client.

**Action space.** Charging power is discretized into four levels (off, low, medium, high: 0, 7.3, 14.7, 22 kW) rather than allowing continuous output. The assignment explicitly permits this simplification, and discrete actions keep DQN applicable without the added complexity of a continuous-control algorithm such as DDPG, which would need substantially more tuning for a 4-way decision that a home charging unit would realistically only support in discrete steps anyway.

**Reward function.** Every slot incurs a charging cost that grows exponentially in power,  $c \cdot e^{p_t/p_{\max}}$ , reflecting that fast charging draws a disproportionate premium on real electricity tariffs (modeled here on average ComEd afternoon supply prices) and stresses the battery more than slow charging. At the end of the window, a shortfall between the realized stochastic demand ( $\mathcal{N}(30, 5^2)$  kWh) and the final SoC is penalized at a coefficient of 100 per kWh missing, deliberately far above the cost scale of charging itself, so that the agent never learns to gamble on running out of energy to save a few cents. Table 16 summarizes the resulting reward per action, evaluated at the reference cost coefficient of the simulation.

Table 16: Charging cost by action, before any shortfall penalty.

| Action | Label  | Power (kW) | Energy per slot (kWh) | Cost this slot |
|--------|--------|------------|-----------------------|----------------|
| 0      | Off    | 0.0        | 0.00                  | 4.00           |
| 1      | Low    | 7.3        | 1.83                  | 5.58           |
| 2      | Medium | 14.7       | 3.67                  | 7.80           |
| 3      | High   | 22.0       | 5.50                  | 10.87          |

**A note on scaling.** An earlier version of the cost function used  $c \cdot e^{p_t}$  directly. Because  $p_t$  ranges up to 22, this exploded numerically and made the reward signal uninformative for the agent. Normalizing the exponent by  $p_{\max}$  keeps the cost in a well-behaved range while preserving the intended convexity: charging faster remains disproportionately expensive, just not to the point of destabilizing training.

## E.2 Agent: Deep Q-Network

We use a Deep Q-Network rather than tabular Q-learning or policy-gradient methods, for three reasons tied to this problem’s structure. First, the action space is small and discrete (four choices), which is exactly the regime Q-learning-based methods are strongest in; policy-gradient methods (REINFORCE, Actor-Critic) add complexity that buys nothing here. Second, the state space is continuous (SoC is a real-valued kWh amount), so tabular Q-learning would require discretizing it and lose precision at the SoC boundaries near the shortfall threshold. Third, DQN’s experience replay lets the agent learn from individual 15-minute transitions rather than waiting for full episodes to complete, which is more sample-efficient than Monte Carlo approaches given the small number of decision points (8) per episode.

The Q-network is a small feedforward network with two hidden layers (12,932 parameters total), trained with a target network and soft updates ( $\tau = 0.0005$ ) for stability, and an  $\varepsilon$ -greedy exploration schedule decaying from 1.0 to 0.001 over training. Training ran for 5,000 episodes (40,000 transitions).

## E.3 Training Behavior

Figure 10 shows the training curves. Reward and shortfall penalty do not decline monotonically: several stretches (around episodes 500 and 1,000) show a spike in average penalty, where the agent temporarily under-charges before correcting. This reflects the exploration-driven trade-off inherent to  $\varepsilon$ -greedy DQN. The agent must occasionally under-charge to learn the shape of the penalty, since the shortfall penalty is only observed at the end of an episode. By the later episodes, the average penalty converges to zero while average final SoC stabilizes above the 30 kWh demand mean, indicating the agent has learned a conservative buffer against demand uncertainty rather than charging to the mean exactly.

## E.4 Evaluation Distributions

## E.5 Example Episode

Figure 12 traces one evaluation episode slot by slot: SoC rises from an initial value in the 10–15 kWh range at 2 p.m. to a final SoC of 44.2 kWh at 4 p.m., against a realized demand draw of 24.4 kWh, leaving a comfortable 19.8 kWh surplus at a total charging cost of 66.33. This single trace should not be read as typical (the realized demand here happened to fall below the

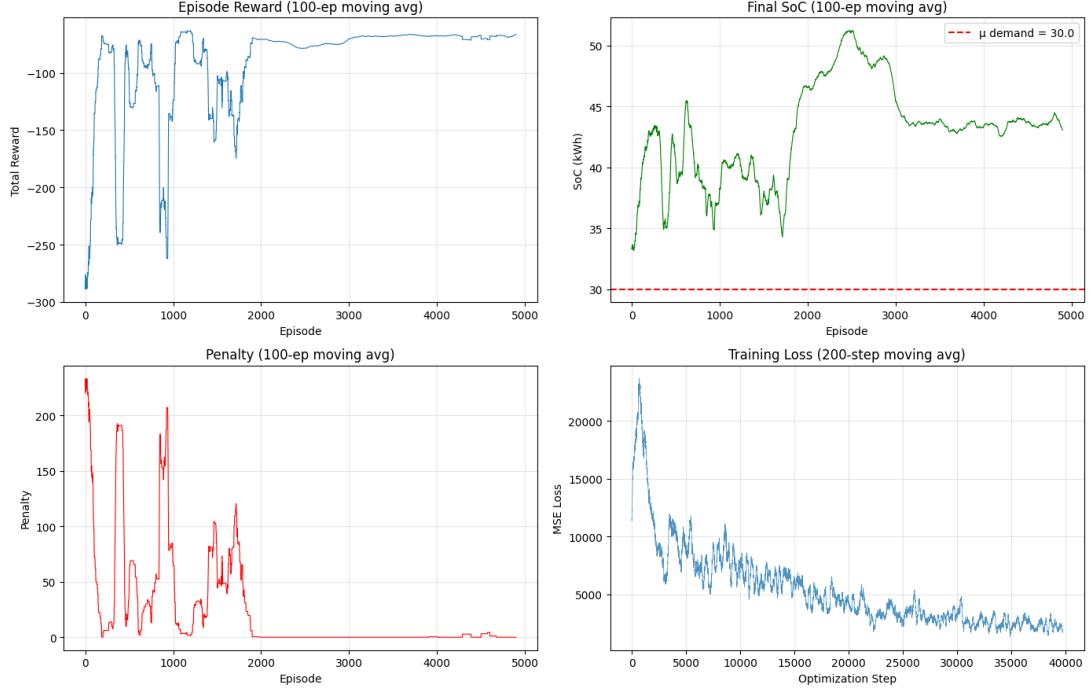


Figure 10: Training curves: episode reward, final SoC, cost and penalty over 5,000 training episodes (100-episode moving average).

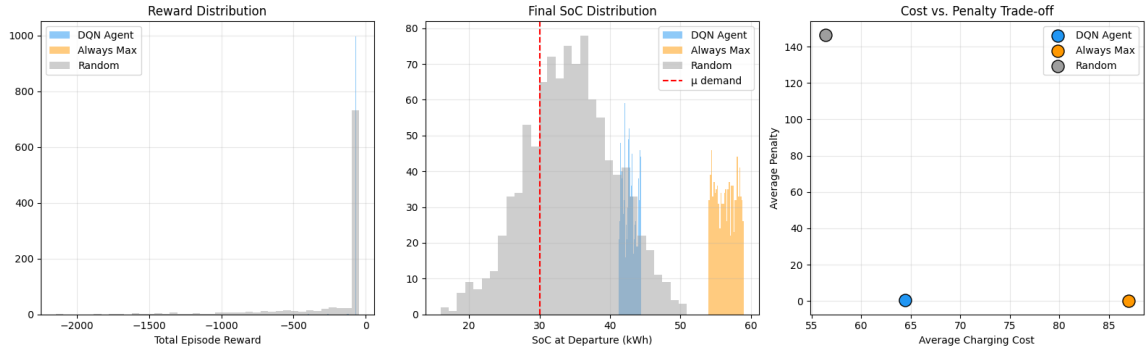


Figure 11: Distributions over 1,000 evaluation episodes: total reward, final SoC, and cost vs. penalty, compared across DQN Agent, Always-Max, and Random.

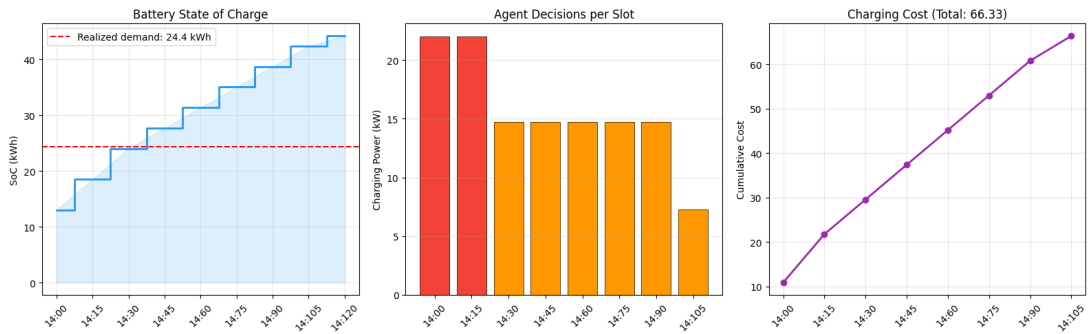


Figure 12: Example evaluation episode: SoC, charging power, and cost per 15-minute slot from 2 p.m. to 4 p.m.

mean), but it illustrates the slot-by-slot decision pattern: higher power early in the window while SoC is low, tapering off as the buffer builds.

## References

- Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65, 126–139. <https://doi.org/10.1016/j.compenvurbsys.2017.05.004>
- Open-Meteo. (2024). Historical weather API [Accessed: 2026-05-10].